High-Concurrency Booking Engine

2026-09-27

High-Concurrency Booking Engine — Schema & Query Submission

Design doc for the BookMyShow-style ticketing backend: all tables with their attributes, example rows, and the executable MySQL SQL for P1 (schema) and P2 (shows-by-theatre-and-date query).

Entities & table structures (P1)

Table	Key attributes
`theatre`	`theatre_id` PK, name, city, address, created_at
`screen`	`screen_id` PK, `theatre_id` FK, name, total_seats — UNIQUE(theatre_id, name)
`movie`	`movie_id` PK, title, language, duration_minutes, genre, release_date
`show`	`show_id` PK, `movie_id` FK, `screen_id` FK, show_date, start_time, end_time — UNIQUE(screen_id, show_date, start_time)
`seat`	`seat_id` PK, `screen_id` FK, row_label, seat_number, seat_type, price — UNIQUE(screen_id, row_label, seat_number)
`customer`	`customer_id` PK, name, email (UNIQUE), phone
`booking`	`booking_id` PK, `show_id` FK, `customer_id` FK, status, total_amount, created_at, expires_at
`booking_seat`	(`booking_id`, `seat_id`) PK, price_at_booking — historical audit of a booking's seats
`show_seat_lock`	(`show_id`, `seat_id`) PK, `booking_id` FK, status, expires_at — the concurrency-safety table
`payment`	`payment_id` PK, `booking_id` FK (UNIQUE), amount, gateway_reference (UNIQUE), status, created_at, updated_at
`webhook_event`	`event_id` PK (from gateway), `payment_id` FK, payload, received_at, processed_at — idempotency guard

Normalization

- **1NF** — every column holds a single atomic value; no repeating groups.
- **2NF** — composite-key tables (`booking_seat`, `show_seat_lock`) have non-key attributes that depend on the *whole* key, not just part of it.
- **3NF / BCNF** — `customer`, `movie`, `theatre` are split into their own tables so `booking`/`show` don't carry transitive attributes (e.g. a customer's email isn't duplicated on every booking row). Every remaining determinant is a candidate key.
- Two deliberate, documented denormalizations that don't break BCNF (each still depends on the whole key of its own row — they're derived/snapshot values, not redundant copies):

`show.end_time` (derivable from `movie.duration_minutes` + `start_time`, kept for cheap read-time queries) and `booking_seat.price_at_booking` (a historical snapshot, since `seat.price` can change after the booking is made).

Concurrency & locking strategy

`show_seat_lock` has `PRIMARY KEY (show_id, seat_id)` — at most one row can ever exist per seat, per show. A second concurrent attempt to hold or book an already-locked seat fails at `INSERT` time with a duplicate-key error, independent of any application-level locking. In the running system this is paired with a Redis distributed lock (`SET NX PX ttl` semantics via Redisson) as a fast-path that fails contention immediately; the database constraint is what actually guarantees no double-booking even if Redis is ever unavailable or stale. Holds carry an `expires_at`; a scheduled reconciliation job releases seats whose hold has expired, so a lost/abandoned hold never blocks a seat forever.

P1: SQL to create the tables

```
-- High-Concurrency Booking Engine schema (MySQL 8, InnoDB, utf8mb4)
-- Normalization notes:
--  * booking_seat.price_at_booking is an intentional historical snapshot, not a
--    redundant copy of seat.price (seat prices can change after a booking is
made).
--  * show.end_time is an intentional denormalization (derivable from
movie.duration
--    + start_time) kept for cheap read-time queries; it does not break BCNF
since it
--    still depends on the whole key (show_id).
--  * show_seat_lock is the concurrency-safety linchpin: its PRIMARY KEY
(show_id,
--    seat_id) means a second concurrent attempt to hold/book the same seat for
the
--    same show fails at INSERT time with a duplicate-key error, independent of
any
--    application-level locking.

CREATE TABLE theatre (
    theatre_id   BIGINT AUTO_INCREMENT PRIMARY KEY,
    name         VARCHAR(150) NOT NULL,
    city         VARCHAR(100) NOT NULL,
    address      VARCHAR(255) NOT NULL,
    created_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE screen (
    screen_id    BIGINT AUTO_INCREMENT PRIMARY KEY,
    theatre_id   BIGINT NOT NULL,
    name         VARCHAR(50) NOT NULL,
    total_seats  INT NOT NULL,
    CONSTRAINT fk_screen_theatre FOREIGN KEY (theatre_id) REFERENCES theatre
(theatre_id),
```

```
    CONSTRAINT uq_screen_theatre_name UNIQUE (theatre_id, name)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE movie (
    movie_id          BIGINT AUTO_INCREMENT PRIMARY KEY,
    title             VARCHAR(200) NOT NULL,
    language          VARCHAR(50) NOT NULL,
    duration_minutes  INT NOT NULL,
    genre             VARCHAR(100),
    release_date      DATE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE `show` (
    show_id      BIGINT AUTO_INCREMENT PRIMARY KEY,
    movie_id     BIGINT NOT NULL,
    screen_id    BIGINT NOT NULL,
    show_date    DATE NOT NULL,
    start_time   TIME NOT NULL,
    end_time     TIME NOT NULL,
    CONSTRAINT fk_show_movie FOREIGN KEY (movie_id) REFERENCES movie (movie_id),
    CONSTRAINT fk_show_screen FOREIGN KEY (screen_id) REFERENCES screen
(screen_id),
    CONSTRAINT uq_show_screen_slot UNIQUE (screen_id, show_date, start_time)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE INDEX idx_show_screen_date_time ON `show` (screen_id, show_date,
start_time);

CREATE TABLE seat (
    seat_id      BIGINT AUTO_INCREMENT PRIMARY KEY,
    screen_id    BIGINT NOT NULL,
    row_label    VARCHAR(5) NOT NULL,
    seat_number  INT NOT NULL,
    seat_type    ENUM('SILVER', 'GOLD', 'PLATINUM') NOT NULL DEFAULT 'SILVER',
    price        DECIMAL(8, 2) NOT NULL,
    CONSTRAINT fk_seat_screen FOREIGN KEY (screen_id) REFERENCES screen
(screen_id),
    CONSTRAINT uq_seat_screen_position UNIQUE (screen_id, row_label, seat_number)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE customer (
    customer_id  BIGINT AUTO_INCREMENT PRIMARY KEY,
    name         VARCHAR(150) NOT NULL,
    email        VARCHAR(150) NOT NULL,
    phone        VARCHAR(20),
    CONSTRAINT uq_customer_email UNIQUE (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE booking (
```

```
    booking_id    BIGINT AUTO_INCREMENT PRIMARY KEY,
    show_id       BIGINT NOT NULL,
    customer_id   BIGINT NOT NULL,
    status        ENUM('HOLD', 'CONFIRMED', 'CANCELLED', 'EXPIRED') NOT NULL
DEFAULT 'HOLD',
    total_amount  DECIMAL(10, 2) NOT NULL,
    created_at    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    expires_at    TIMESTAMP NOT NULL,
    CONSTRAINT fk_booking_show FOREIGN KEY (show_id) REFERENCES `show` (show_id),
    CONSTRAINT fk_booking_customer FOREIGN KEY (customer_id) REFERENCES customer
(customer_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE INDEX idx_booking_show ON booking (show_id);
CREATE INDEX idx_booking_status_expiry ON booking (status, expires_at);

-- Historical/audit record of which seats belong to which booking.
CREATE TABLE booking_seat (
    booking_id        BIGINT NOT NULL,
    seat_id           BIGINT NOT NULL,
    price_at_booking  DECIMAL(8, 2) NOT NULL,
    PRIMARY KEY (booking_id, seat_id),
    CONSTRAINT fk_booking_seat_booking FOREIGN KEY (booking_id) REFERENCES
booking (booking_id),
    CONSTRAINT fk_booking_seat_seat FOREIGN KEY (seat_id) REFERENCES seat
(seat_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Live, concurrency-safe reservation state: at most one row per (show_id,
seat_id)
-- ever exists, so a duplicate INSERT for an already-held/booked seat fails
atomically.
CREATE TABLE show_seat_lock (
    show_id     BIGINT NOT NULL,
    seat_id     BIGINT NOT NULL,
    booking_id  BIGINT NOT NULL,
    status      ENUM('HOLD', 'CONFIRMED') NOT NULL DEFAULT 'HOLD',
    expires_at  TIMESTAMP NULL,
    PRIMARY KEY (show_id, seat_id),
    CONSTRAINT fk_lock_show FOREIGN KEY (show_id) REFERENCES `show` (show_id),
    CONSTRAINT fk_lock_seat FOREIGN KEY (seat_id) REFERENCES seat (seat_id),
    CONSTRAINT fk_lock_booking FOREIGN KEY (booking_id) REFERENCES booking
(booking_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE INDEX idx_lock_status_expiry ON show_seat_lock (status, expires_at);

CREATE TABLE payment (
    payment_id         BIGINT AUTO_INCREMENT PRIMARY KEY,
```

```
    booking_id         BIGINT NOT NULL,
    amount             DECIMAL(10, 2) NOT NULL,
    gateway_reference  VARCHAR(100) NOT NULL,
    status             ENUM('PENDING', 'SUCCESS', 'FAILED') NOT NULL DEFAULT
'PENDING',
    created_at         TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at         TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    CONSTRAINT fk_payment_booking FOREIGN KEY (booking_id) REFERENCES booking
(booking_id),
    CONSTRAINT uq_payment_booking UNIQUE (booking_id),
    CONSTRAINT uq_payment_gateway_reference UNIQUE (gateway_reference)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Idempotency guard: event_id comes from the payment gateway, so a duplicate
-- webhook delivery fails this INSERT and is skipped instead of reprocessed.
CREATE TABLE webhook_event (
    event_id      VARCHAR(100) PRIMARY KEY,
    payment_id    BIGINT NOT NULL,
    payload       JSON NOT NULL,
    received_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    processed_at  TIMESTAMP NULL,
    CONSTRAINT fk_webhook_payment FOREIGN KEY (payment_id) REFERENCES payment
(payment_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Example rows

theatre

theatre_id	name	city	address
1	PVR Forum Mall	Bengaluru	Forum Mall, Hosur Road, Koramangala

screen

screen_id	theatre_id	name	total_seats
1	1	Screen 1	20
2	1	Screen 2	20

movie

movie_id	title	language	duration_minutes	genre	release_date
1	Galaxy Raiders	English	148	Sci-Fi	2026-08-15
2	Monsoon Melody	Hindi	132	Drama	2026-09-01

show

show_id	movie_id	screen_id	show_date	start_time	end_time
1	1	1	2026-09-27	10:00:00	12:28:00
2	2	1	2026-09-27	15:00:00	17:12:00
11	2	2	2026-09-27	11:00:00	13:12:00

seat

seat_id	screen_id	row_label	seat_number	seat_type	price
1	1	A	1	GOLD	220.00
6	1	B	1	PLATINUM	320.00

customer

customer_id	name	email	phone
1	Asha Rao	asha.rao@example.com	+919845012345
2	Vikram Shah	vikram.shah@example.com	+919845098765

booking

booking_id	show_id	customer_id	status	total_amount	expires_at
1	1	1	CON-FIRMED	440.00	2026-09-26 09:10:00
2	1	2	HOLD	320.00	(now + 10 min)

booking_seat

booking_id	seat_id	price_at_booking
1	1	220.00
1	2	220.00

show_seat_lock

show_id	seat_id	booking_id	status	expires_at
1	1	1	CONFIRMED	NULL
1	6	2	HOLD	(now + 10 min)

payment

payment_id	booking_id	amount	gateway_reference	status
1	1	440.00	pay_mock_0001	SUCCESS

webhook_event

event_id	payment_id	payload	processed_at
evt_mock_0001	1	{"event":"payment.succeeded",2026}	09-26 09:01:30

Full sample-data INSERT statements

```
INSERT INTO theatre (theatre_id, name, city, address) VALUES
    (1, 'PVR Forum Mall', 'Bengaluru', 'Forum Mall, Hosur Road, Koramangala');

INSERT INTO screen (screen_id, theatre_id, name, total_seats) VALUES
    (1, 1, 'Screen 1', 20),
    (2, 1, 'Screen 2', 20);

INSERT INTO movie (movie_id, title, language, duration_minutes, genre,
release_date) VALUES
    (1, 'Galaxy Raiders', 'English', 148, 'Sci-Fi', '2026-08-15'),
    (2, 'Monsoon Melody', 'Hindi', 132, 'Drama', '2026-09-01'),
    (3, 'The Last Wicket', 'English', 121, 'Sports', '2026-09-20');

INSERT INTO `show` (show_id, movie_id, screen_id, show_date, start_time,
end_time) VALUES
    (1,  1, 1, '2026-09-27', '10:00:00', '12:28:00'),
    (2,  2, 1, '2026-09-27', '15:00:00', '17:12:00'),
    (11, 2, 2, '2026-09-27', '11:00:00', '13:12:00'),
    (12, 3, 2, '2026-09-27', '19:00:00', '21:01:00');
    -- full 18-show set (all 2 screens x next 7 dates) is in
    -- src/main/resources/db/migration/V2__seed_sample_data.sql

INSERT INTO seat (seat_id, screen_id, row_label, seat_number, seat_type, price)
VALUES
    (1, 1, 'A', 1, 'GOLD', 220.00),
    (2, 1, 'A', 2, 'GOLD', 220.00),
    (6, 1, 'B', 1, 'PLATINUM', 320.00);
    -- full 20-seat set (10 per screen) is in V2__seed_sample_data.sql

INSERT INTO customer (customer_id, name, email, phone) VALUES
    (1, 'Asha Rao', 'asha.rao@example.com', '+919845012345'),
    (2, 'Vikram Shah', 'vikram.shah@example.com', '+919845098765');

-- Confirmed booking: Asha booked seats A1 & A2 for show 1, paid successfully.
INSERT INTO booking (booking_id, show_id, customer_id, status, total_amount,
created_at, expires_at) VALUES
    (1, 1, 1, 'CONFIRMED', 440.00, '2026-09-26 09:00:00', '2026-09-26 09:10:00');
```

```
INSERT INTO booking_seat (booking_id, seat_id, price_at_booking) VALUES
    (1, 1, 220.00),
    (1, 2, 220.00);

INSERT INTO show_seat_lock (show_id, seat_id, booking_id, status, expires_at)
VALUES
    (1, 1, 1, 'CONFIRMED', NULL),
    (1, 2, 1, 'CONFIRMED', NULL);

INSERT INTO payment (payment_id, booking_id, amount, gateway_reference, status,
created_at, updated_at) VALUES
    (1, 1, 440.00, 'pay_mock_0001', 'SUCCESS', '2026-09-26 09:01:00', '2026-09-26
09:01:30');

INSERT INTO webhook_event (event_id, payment_id, payload, received_at,
processed_at) VALUES
    ('evt_mock_0001', 1, JSON_OBJECT('event', 'payment.succeeded',
'gateway_reference', 'pay_mock_0001', 'amount', 440.00), '2026-09-26 09:01:15',
'2026-09-26 09:01:30');

-- Pending hold: Vikram is mid-checkout on seat B1 for show 1 (demo of the HOLD
state).
INSERT INTO booking (booking_id, show_id, customer_id, status, total_amount,
created_at, expires_at) VALUES
    (2, 1, 2, 'HOLD', 320.00, NOW(), DATE_ADD(NOW(), INTERVAL 10 MINUTE));

INSERT INTO booking_seat (booking_id, seat_id, price_at_booking) VALUES
    (2, 6, 320.00);

INSERT INTO show_seat_lock (show_id, seat_id, booking_id, status, expires_at)
VALUES
    (1, 6, 2, 'HOLD', DATE_ADD(NOW(), INTERVAL 10 MINUTE));
```

P2: list all shows on a given date at a given theatre, with show timings

```
SELECT s.show_id, m.title AS movie_title, sc.name AS screen_name,
       s.show_date, s.start_time, s.end_time
FROM `show` s
JOIN screen sc ON s.screen_id = sc.screen_id
JOIN movie m ON s.movie_id = m.movie_id
WHERE sc.theatre_id = ? AND s.show_date = ?
ORDER BY s.start_time;
```

Supported by `idx_show_screen_date_time (screen_id, show_date, start_time)` on `show` plus the foreign-key index on `screen.theatre_id`. Run against the sample data above (`theatre_id = 1`, `show_date = '2026-09-27'`):

show_id	movie_title	screen_name	show_date	start_time	end_time
1	Galaxy Raiders	Screen 1	2026-09-27	10:00:00	12:28:00
11	Monsoon Melody	Screen 2	2026-09-27	11:00:00	13:12:00
2	Monsoon Melody	Screen 1	2026-09-27	15:00:00	17:12:00
12	The Last Wicket	Screen 2	2026-09-27	19:00:00	21:01:00

Both queries above were run and verified directly against a local MySQL 8 instance during development.